Claude Code 入門（ChatGPT ユーザー向け）

aoki-taquan

2026 年 03 月 24 日

目次

1 Claude Code とは何か — ChatGPT との違い 4
1.1 ChatGPT でやっていたこと 4
1.2 2 つの AI の違い 4
1.3 具体的なイメージ 4
1.4 エージェント型 AI とは 4
1.5 安全性について 5
1.6 まとめ 5
2 準備 6
2.1 必要なもの 6
2.2 ターミナルとは 6
2.2.1 ターミナルの開き方 6
2.2.2 基本的なターミナル操作 6
2.3 アカウントの準備 7
2.4 インストール 7
2.4.1 macOS / Linux 7
2.4.2 Windows（PowerShell） 7
2.4.3 インストール完了の確認 7
2.5 ログイン 7
2.6 ブラウザ版（インストール不要） 7
3 最初の一歩 — 起動と基本操作 9
3.1 Claude Code を起動する 9
3.2 ChatGPT と同じように話しかける 9
3.3 会話の流れ 9
3.4 ファイルを指定して話しかける 10
3.5 終了する 10
3.6 前回の会話を続ける 10
3.7 会話をリセットする 10
3.8 確認を求められたら 10
3.9 最初に試してみること 11
4 ファイルを触ってもらう 12
4.1 Claude Code にできること 12
4.2 ファイルを読んで教えてもらう 12
4.3 コードを修正してもらう 12
4.3.1 変更内容の見方 12
4.4 新しいファイルを作ってもらう 12
4.5 複数ファイルをまとめて変更してもらう 13
4.6 コマンドを実行してもらう 13
4.7 バグを修正してもらう 13
4.8 変更を承認・拒否する 13

4.8.1 Plan モード（読み取りのみ） 13
4.8.2 権限モードの切り替え 13
5 実践ユースケース 15
5.1 新機能を追加する 15
5.2 バグを直す 15
5.3 コードを理解する 15
5.4 リファクタリング（コードを整理する） 15
5.5 テストを書く 16
5.6 ドキュメントを書く 16
5.7 コードレビューをしてもらう 16
5.8 依存関係を調べる 16
5.9 翻訳・国際化 16
5.10 Git 操作 16
6 便利な使い方 17
6.1 CLAUDE.md — プロジェクトのルールを伝える 17
6.2 Plan モード — 作業前に計画を確認する 17
6.3 キーボードショートカット 18
6.3.1 よく使うもの 18
6.3.2 画像を貼り付ける 18
6.4 /btw — 脱線した質問をする 18
6.5 モデルを使い分ける 18
6.6 じっくり考えさせる（Effort Level） 19
6.7 よくある失敗とその対処 19
6.7.1 「変更が多すぎる」と感じたとき 19
6.7.2 意図と違う変更をされたとき 19
6.7.3 会話が長くなって忘れられてきたとき 19
6.8 VS Code から使う 19
6.9 上達のコツ 19
6.9.1 具体的に指示する 19
6.9.2 まず Plan モードで確認する 20
6.9.3 CLAUDE.md を育てる 20

1 Claude Code とは何か — ChatGPT との違い

1.1 ChatGPT でやっていたこと

ChatGPT を使ってコードを書いてもらった経験はありますか？「このコードを修正して」「Python でスクレイピングを書いて」と頼むと、コードが返ってきます。でも、その後は自分でコピペして、ファイルに貼り付けて…という手順が必要でした。

Claude Code は、**その「コピペ」以降も全部やってくれる** AI です。

1.2 2 つの AI の違い

	ChatGPT	Claude Code
操作する場所	ブラウザ（Web）	ターミナル（黒い画面）
コードの扱い	テキストとして返すだけ	実際にファイルに書いて保存
作業の範囲	「答え」を出す	「作業」を実行する
エラーが出たら	別途聞き直す	自分で確認して修正まで行う
ファイルの確認	自分で貼り付けて見せる必要がある	プロジェクト全体を自分で読む

1.3 具体的なイメージ

ChatGPT に「login.py にバリデーションを追加して」と頼んだとします。

ChatGPT の場合：

1. コードのスニペットが返ってくる
2. 自分でコピーして login.py を開く
3. 該当箇所に貼り付ける
4. 動くか確認する
5. エラーが出たら再度 ChatGPT に貼り付けて聞く

Claude Code の場合：

1. `login.py にバリデーションを追加して` と入力するだけ
2. Claude Code が login.py を自分で読んで変更箇所を判断する
3. ファイルを直接編集する（変更前に確認を求めてくれる）
4. テストがあれば実行して動作を確認する
5. エラーがあれば自分で修正する

1.4 エージェント型 AI とは

Claude Code は「エージェント（代理人）型」と呼ばれます。ChatGPT が「質問に答える AI」なら、Claude Code は「代わりに仕事をしてくれる AI」です。

- 自分でファイルを開いて読む

- 何をすべきか判断する
- 実際に操作を実行する
- 結果を確認して問題があれば直す

1.5 安全性について

「勝手にファイルを変えられたら困る」と心配になるかもしれません。Claude Code は必ず**事前に確認を求めます**。

ファイルを編集する前、コマンドを実行する前に、「こういう変更をしていいですか？」と聞いてきます。 OK しなければ何もしません。

1.6 まとめ

Claude Code は「自分の代わりに開発作業をこなしてくれる AI」です。ChatGPT で「答えを教えてもらって自分で作業する」のに慣れている方は、最初は少し戸惑うかもしれませんが、慣れると手放せなくなります。

2 準備

2.1 必要なもの

Claude Code を使い始めるために必要なものは 2 つだけです：

1. **Anthropic アカウント**（Claude.ai Max プランなど）
2. **ターミナル**（最初から入っています）

2.2 ターミナルとは

Claude Code はターミナルから操作します。ターミナルとは、コンピュータに文字でコマンドを入力する画面です。 黒い（または暗い）背景に文字が並んでいるあれです。

難しそうに見えますが、使うのは「コマンドを打って Enter を押す」だけです。

2.2.1 ターミナルの開き方

OS	開き方
macOS	`Cmd+Space` → 「ターミナル」と入力 → Enter
Windows	`Win+R` → `powershell` と入力 → Enter
Linux	`Ctrl+Alt+T` またはアプリ一覧から「端末」

開くと `$` や `%` や `>` で終わる行が表示されます。これが「入力待ち」の状態です。

2.2.2 基本的なターミナル操作

Claude Code を使うために最低限知っておくべきことだけ覚えましょう。

フォルダを移動する（cd）：

```
cd Desktop          # Desktop フォルダに移動
cd my-project       # my-project フォルダに移動
cd ..               # 一つ上のフォルダに移動
```

現在地を確認する（pwd）：

```
pwd
```

フォルダの中身を見る（ls）：

```
ls
```

これだけで Claude Code は使えます。

2.3 アカウントの準備

Claude Code を使うには Anthropic のアカウントが必要です。

現在対応しているプランは以下の通りです：

- **Claude.ai Max プラン**（個人向け、月額課金）
- **Team / Enterprise プラン**（チーム・法人向け）
- **Anthropic Console アカウント**（API 利用者向け）

ChatGPT Plus のような感覚で、有料プランへの加入が必要です。 まだの方は claude.ai から登録してください。

2.4 インストール

ターミナルを開いて、以下のコマンドをコピペして Enter を押します。

2.4.1 macOS / Linux

```
curl -fsSL https://claude.ai/install.sh | bash
```

2.4.2 Windows（PowerShell）

```
irm https://claude.ai/install.ps1 | iex
```

コマンドを実行するとインストールが自動で始まります。 1〜2 分ほど待てば完了します。

2.4.3 インストール完了の確認

```
claude --version
```

バージョン番号が表示されれば成功です。

2.5 ログイン

インストールできたら、Claude Code を起動してみます：

```
claude
```

初回起動時にブラウザが自動的に開きます。 Anthropic アカウントでログインすれば準備完了です。 ChatGPT にログインするのと同じ感覚です。

2.6 ブラウザ版（インストール不要）

「まず試してみたい」という場合は、ブラウザ版があります。 claude.ai にアクセスして Claude Code を有効にすると、 インストールなしで同じように使えます。

慣れてきたらローカルにインストールすることをおすすめします。

3 最初の一歩 — 起動と基本操作

3.1 Claude Code を起動する

まず作業したいプロジェクトのフォルダに移動して起動します。

```
cd my-project    # 作業フォルダに移動
claude           # Claude Code を起動
```

`>` が表示されたら準備完了です。ここが入力欄です。

3.2 ChatGPT と同じように話しかける

基本的な使い方は ChatGPT と変わりません。日本語で話しかけるだけです。

```
> このプロジェクトの構成を説明して
> README.md の内容を要約して
> このエラーの意味を教えて
```

違いは、Claude Code が**実際にファイルを読んで答えてくれる**点です。ファイルの内容を手動でコピペして貼り付ける必要はありません。

3.3 会話の流れ

Claude Code との基本的なやりとりのイメージです。

プロジェクト構成を確認してもらう：

```
> このプロジェクトの構成を教えて

Claude: プロジェクトを確認します…
（ファイルを自動的に確認）
このプロジェクトは Node.js + Express で構成されており、
src/ 以下にルーティングとコントローラが…
```

コードを修正してもらう：

```
> login.ts の認証処理にエラーハンドリングを追加して

Claude: login.ts を確認します…
（ファイルを読む）
以下の変更を加えます：
（差分を表示）
実行してよいですか？ [Yes/No]
```

「Yes」と答えると、実際にファイルが変更されます。

3.4 ファイルを指定して話しかける

`@` を使うと特定のファイルを参照できます：

```
> @src/auth.ts のバグを修正して
> @README.md を英語に翻訳して
```

`@` の後にファイル名を入力するとオートコンプリートされます。ChatGPT にコードをコピペして「これを直して」と頼む代わりに、`@` で参照するだけです。

3.5 終了する

Claude Code を終了するには：

- `/exit` と入力する
- `Ctrl+C` を 2 回押す

3.6 前回の会話を続ける

ChatGPT で過去のチャット履歴を開くように、前回の作業を再開できます：

```
claude -c       # 前回の会話を再開
claude --resume # 過去の会話一覧から選んで再開
```

3.7 会話をリセットする

話題を切り替えたいときは：

```
/clear
```

ChatGPT で「新しいチャット」を開くのと同じです。

3.8 確認を求められたら

Claude Code はファイルを変更したりコマンドを実行する前に必ず確認してきます。

```
このファイルを変更してよいですか？
  Allow once（今回だけ）
  Allow for this session（このセッション中は常に）
  Always allow（常に許可）
  Deny（拒否）
```

慣れるまでは「Allow once」で OK です。「Always allow」にすると同じ種類の操作を毎回確認なしで進めます。

3.9 最初に試してみること

以下は Claude Code を触り始めるのにちょうどいいお題です：

```
> このプロジェクトの構成を説明して
> src/ 以下のファイル一覧を見せて
> package.json の内容を説明して
> テストを実行して
```

まずは**読んで教えてもらう**ことから慣れると、次第に**書いて変更してもらう**ことへのハードルも下がります。

4 ファイルを触ってもらう

4.1 Claude Code にできること

Claude Code は以下の操作を自分でこなします：

- **読む**：ファイルの内容を確認する
- **検索する**：条件に合うファイルやコードを探す
- **編集する**：既存のファイルを変更する
- **作成する**：新しいファイルを作る
- **実行する**：コマンドやテストを走らせる

4.2 ファイルを読んで教えてもらう

```
> src/main.ts の内容を見せて
> @config.json の設定を説明して
> このプロジェクトのディレクトリ構成を教えて
```

ChatGPT と違い、ファイルの内容を手でコピペする必要はありません。 Claude Code が自分でファイルを開いて読みます。

4.3 コードを修正してもらう

```
> handleLogin 関数にエラーハンドリングを追加して
> @src/utils.ts の calculateTotal 関数を最適化して
> コメントを日本語で追加して
```

変更内容は差分（どの行が変わるか）で表示されます。 確認して OK なら承認、気に入らなければ却下できます。

4.3.1 変更内容の見方

```
- const result = process(data)      ← 削除される行（赤）
+ const result = process(data) ?? [] ← 追加される行（緑）
```

ChatGPT でコードをもらってから「どこをどう変えればいいか」を 自分で考える必要がなくなります。

4.4 新しいファイルを作ってもらう

```
> src/utils/validation.ts にメールアドレス検証の関数を作って
> テスト用のサンプルデータファイルを作って
```

ファイルの中身を考えて書くところまで自動でやってくれます。

4.5 複数ファイルをまとめて変更してもらう

```
> UserService を UserRepository と UserUseCase に分割して
> 全ファイルの console.log をロガーに置き換えて
```

ChatGPT では 1 ファイルずつコピペして「次はこれも修正して」という手間がかかりましたが、Claude Code はプロジェクト全体を把握して、関連するファイルを一括で変更します。

4.6 コマンドを実行してもらう

```
> テストを実行して
> ビルドして
> npm install して
```

コマンドの結果（エラーや出力）も Claude Code が確認して、問題があれば自分で直そうとします。

4.7 バグを修正してもらう

エラーが出たとき、エラーメッセージをそのまま貼り付けるだけで対応してくれます：

```
> TypeError: Cannot read property 'id' of undefined
> というエラーが出ているので修正して
```

ChatGPT にエラーを貼り付けて「原因は？」「修正方法は？」と 何往復もするところを、Claude Code は一気に解決しようとします。

4.8 変更を承認・拒否する

Claude Code は変更前に必ず確認します。慎重に使いたい場合は毎回確認できます。

4.8.1 Plan モード（読み取りのみ）

ファイルを変更せず、作業計画だけ表示してもらうモードです：

```
claude --permission-mode plan
```

「何をするつもりなのか」を先に確認してから作業を進められます。大きな変更をお願いするときに便利です。

4.8.2 権限モードの切り替え

`Shift+Tab` で権限モードを切り替えられます：

モード	動作
`default`	操作ごとに確認を求める（初心者向け）
`plan`	読み取りのみ。何をするか計画を先に見せる
`acceptEdits`	ファイル編集は自動 OK。コマンド実行は確認

最初は `default` のまま使い、慣れてきたら `acceptEdits` にすると快適になります。

5 実践ユースケース

ChatGPT でやっていたことが、Claude Code でどう変わるかを具体的に見てみましょう。

5.1 新機能を追加する

ChatGPT の場合：「パスワードリセット機能を実装するコードを書いて」→ コードが返ってくる → 自分でファイルに組み込む

Claude Code の場合：

```
> パスワードリセット機能を追加して。
> メールで確認コードを送る方式で。
```

Claude Code がプロジェクト構成を確認して、適切なファイルに実装します。

5.2 バグを直す

ChatGPT の場合：エラーコードをコピペ → 原因を教えてもらう → 修正コードを教えてもらう → 自分で直す

Claude Code の場合：

```
> 以下のエラーが出ています：
> TypeError: Cannot read property 'name' of undefined
>   at UserCard (src/components/UserCard.tsx:15)
> 修正して
```

Claude Code が該当ファイルを読んで、直接修正します。

5.3 コードを理解する

引き継いだコードや他人のコードを読む場合も ChatGPT と同じ感覚で使えます：

```
> このプロジェクトのログイン処理がどのように動いているか説明して
> @src/auth/token.ts の処理フローを図解して
> このコードのセキュリティ上の問題点はある？
```

ファイルを手動で貼り付けなくても、Claude Code が自分で読んで答えます。

5.4 リファクタリング（コードを整理する）

```
> src/api/users.ts のコードが長くなりすぎているので整理して
> 重複しているコードを共通化して
> TypeScript の型定義が甘い箇所を厳密にして
```

影響範囲を確認してから変更してくれるので、思わぬ箇所が壊れるリスクが下がります。

5.5 テストを書く

```
> @src/utils/validation.ts の関数に対するテストを書いて
> カバレッジが低いファイルのテストを追加して
```

既存のコードを読んだうえで、適切なテストケースを考えて書いてくれます。

5.6 ドキュメントを書く

```
> このプロジェクトの README を書いて
> @src/api/ の各エンドポイントの API ドキュメントを書いて
> 関数にコメントを追加して
```

コードを読んで内容を把握したうえで、適切なドキュメントを生成します。

5.7 コードレビューをしてもらう

```
> 今の変更内容をレビューして
> セキュリティ的に問題のある箇所はある？
> このコードのパフォーマンス上の問題点を指摘して
```

5.8 依存関係を調べる

```
> package.json の依存パッケージに脆弱性があるか確認して
> 使われていない import を探して削除して
> このライブラリはどこで使われている？
```

5.9 翻訳・国際化

```
> src/i18n/ja.json を見て、同じ構造の en.json を英語で作って
> コード内のハードコードされた日本語テキストを一覧にして
```

5.10 Git 操作

```
> 今の変更をコミットして。コミットメッセージは日本語で
> この変更の内容を説明するプルリクエストを作って
```

Claude Code が変更内容を確認して適切なコミットメッセージを考えてくれます。

6 便利な使い方

6.1 CLAUDE.md — プロジェクトのルールを伝える

プロジェクトのルートに `CLAUDE.md` というファイルを置くと、Claude Code がそのプロジェクトで作業するときのルールを覚えてくれます。

ChatGPT で毎回「このプロジェクトは〇〇という技術で…」と説明していた部分が不要になります。

```
# CLAUDE.md

## プロジェクト概要
TypeScript + React の Todo アプリ

## 開発規約
- コンポーネントは関数コンポーネントで書く
- スタイルは Tailwind CSS を使う
- テストは Vitest で書く

## よく使うコマンド
- 開発サーバー: npm run dev
- テスト: npm test
- ビルド: npm run build
```

`/init` と入力すると、Claude Code が現在のプロジェクトを見て CLAUDE.md を自動生成してくれます。

6.2 Plan モード — 作業前に計画を確認する

大きな変更を依頼するときは、先に計画だけ見てから判断できます：

```
claude --permission-mode plan
```

または起動後に `Shift+Tab` でモードを切り替えられます。

Plan モードでは Claude Code はファイルを読むことはしますが、変更はしません。「こういう手順で変更します」という計画を提示してくれるので、OK なら通常モードに切り替えて実行します。

6.3 キーボードショートカット

6.3.1 よく使うもの

ショートカット	動作
`Enter`	メッセージを送信
`Shift+Enter`	改行（長い指示を書くとき）
`Ctrl+C`	処理を中断 / 2 回押すと終了
`上矢印`	直前のプロンプトを呼び出す
`Esc × 2`	直前の操作を巻き戻す
`Shift+Tab`	権限モードを切り替える

6.3.2 画像を貼り付ける

スクリーンショットや図をそのまま質問に使えます：

```
Ctrl+V　（画像をクリップボードからそのまま貼り付け）
```

「このエラー画面を修正して」とスクリーンショットを貼るだけで対応してくれます。

6.4 /btw — 脱線した質問をする

作業の途中でちょっと別のことを聞きたいとき、`/btw` を使うと 会話の流れに影響せず質問できます：

```
/btw TypeScript の interface と type の違いって何？
```

6.5 モデルを使い分ける

Claude Code には複数のモデルがあります。ChatGPT の GPT-4o と GPT-3.5 の使い分けに近いです。

モデル	向いている場面
Sonnet（デフォルト）	普段使い。速度とクオリティのバランスが良い
Opus	難しい問題、複雑な設計や推論が必要なとき
Haiku	シンプルな質問、コスト重視のとき

セッション中にいつでも切り替えられます：

```
/model opus
```

6.6 じっくり考えさせる（Effort Level）

複雑な問題のときは、Claude Code によく考えさせることができます：

```
claude --effort high      # 起動時に指定
/effort high              # セッション中に切り替え
```

6.7 よくある失敗とその対処

6.7.1 「変更が多すぎる」と感じたとき

大きすぎる作業を一度に頼んだとき、変更範囲が想定より広がることがあります。

対処：タスクを細かく分けて依頼する

```
❌ 「このアプリ全体をリファクタリングして」
✓  「まず src/api/ だけを整理して」
✓  「次に src/components/ の重複を取り除いて」
```

6.7.2 意図と違う変更をされたとき

`Esc × 2` で直前の操作を巻き戻せます。 または `/clear` でやり直して、もっと詳しく指示します。

6.7.3 会話が長くなって忘れられてきたとき

ChatGPT でも起きる「長い会話で前の内容を忘れる」現象と同じです。 `/compact` で会話を要約・圧縮できます。

重要な前提は CLAUDE.md に書いておくと毎回伝える必要がなくなります。

6.8 VS Code から使う

VS Code に Claude Code の拡張機能をインストールすると、 エディタを離れずに使えます。

`Cmd+Shift+X`（Windows: `Ctrl+Shift+X`）で拡張機能パネルを開いて 「Claude Code」を検索してインストールするだけです。

`Cmd+Esc`（Windows: `Ctrl+Esc`）でエディタと Claude Code のパネルを行き来できます。

6.9 上達のコツ

6.9.1 具体的に指示する

```
❌ 「このコードをよくして」
✓  「handleSubmit に null チェックを追加して、null の場合は "入力してください" を返して」
```

ChatGPT に良い回答をもらうときと同じで、具体的なほど期待通りの結果になります。

6.9.2 まず Plan モードで確認する

大きな変更は先に計画を見てから実行するクセをつけると安心です。

6.9.3 CLAUDE.md を育てる

プロジェクトの規約やよく使うコマンドを CLAUDE.md に書いていくと、 毎回の説明が減って作業がスムーズになります。